

Matrix Chain Multiplication using Dynamic Programming

Description

Matrix Chain Multiplication is a Dynamic Programming problem used to determine the most efficient way to multiply a sequence of matrices. The program does not actually multiply the matrices. Instead, it finds the minimum number of scalar multiplications required by choosing the best order of matrix multiplication.

Objectives

- Implement Matrix Chain Multiplication using Dynamic Programming.
- Accept matrix dimensions from the user.
- Calculate the minimum number of scalar multiplications.
- Display time and space complexity.
- Measure and display actual execution time.

Requirements

- Python 3.x
- No external libraries are required.

File Structure

```
Matrix-Chain-Multiplication/  
├──  
├── matrix_chain_multiplication.py  
└── README.md
```

Complete Python Code

```
import time  
  
def matrix_chain_order(p):  
    n = len(p) - 1  
  
    # Create DP table  
    dp = [[0 for _ in range(n)] for _ in range(n)]  
  
    # length is the chain length  
    for length in range(2, n + 1):  
        for i in range(n - length + 1):  
            j = i + length - 1  
  
            dp[i][j] = float('inf')  
  
            # Try every possible split  
            for k in range(i, j):  
                cost = (  
                    dp[i][k]  
                    + dp[k + 1][j]  
                    + p[i] * p[k + 1] * p[j + 1]  
                )  
  
                if cost < dp[i][j]:  
                    dp[i][j] = cost  
  
    return dp[0][n - 1]  
  
print("Matrix Chain Multiplication using Dynamic Programming")  
  
n = int(input("Enter the number of matrices: "))  
  
p = []  
print("\nEnter the dimensions of the matrices.")
```

```

rows = int(input("Enter rows of Matrix 1: "))
cols = int(input("Enter columns of Matrix 1: "))

p.append(rows)
p.append(cols)

previous_cols = cols

for i in range(2, n + 1):
    rows = previous_cols
    cols = int(input(f"Enter columns of Matrix {i}: "))

    p.append(cols)
    previous_cols = cols

print("\nMatrix dimensions:")

for i in range(n):
    print(f"Matrix {i + 1}: {p[i]} x {p[i + 1]}")

start_time = time.perf_counter()

minimum_cost = matrix_chain_order(p)

end_time = time.perf_counter()

execution_time = end_time - start_time

print("\nMinimum number of scalar multiplications:",
      minimum_cost)

print("Time Complexity:  $O(n^3)$ ")

print("Space Complexity:  $O(n^2)$ ")

print("Execution Time: {:.10f} seconds".format(execution_time))

```

How to Run

1. Make sure Python 3.x is installed.
2. Save the program as **matrix_chain_multiplication.py**.
3. Open a terminal in the same folder.
4. Run: `python matrix_chain_multiplication.py`

Input

The program asks for the number of matrices, the rows and columns of the first matrix, and the columns of each remaining matrix. For valid multiplication, the columns of Matrix i must equal the rows of Matrix $i+1$.

Sample Input

```
Enter the number of matrices: 4

Enter the dimensions of the matrices.
Enter rows of Matrix 1: 10
Enter columns of Matrix 1: 20
Enter columns of Matrix 2: 30
Enter columns of Matrix 3: 40
Enter columns of Matrix 4: 30
```

Sample Output

```
Matrix dimensions:
Matrix 1: 10 x 20
Matrix 2: 20 x 30
Matrix 3: 30 x 40
Matrix 4: 40 x 30

Minimum number of scalar multiplications: 30000
Time Complexity:  $O(n^3)$ 
Space Complexity:  $O(n^2)$ 
Execution Time: 0.0000123400 seconds
```

Dynamic Programming Approach

The program uses a two-dimensional DP table, $dp[i][j]$, where $dp[i][j]$ represents the minimum number of scalar multiplications required to multiply matrices i through j . The algorithm tries every possible split position and stores the minimum cost.

$$\text{Cost} = dp[i][k] + dp[k+1][j] + p[i] \times p[k+1] \times p[j+1]$$

Time Complexity

$O(n^3)$ — There are three nested loops for chain length, starting position, and possible split position.

Space Complexity

$O(n^2)$ — The algorithm stores results in an $n \times n$ DP table.

Execution Time

Python's `time.perf_counter()` is used to measure the actual execution time of the Dynamic Programming function. The measured value may vary depending on the computer and system load.

Features

Feature	Included
User Input	Yes
Dynamic Programming	Yes
Minimum Multiplication Cost	Yes

Time Complexity	Yes
Space Complexity	Yes
Execution Time	Yes
Python	Yes

Advantages

- Reduces repeated calculations using Dynamic Programming.
- Finds the optimal multiplication order.
- Accepts different matrix dimensions from the user.
- Measures actual program execution time.
- Easy to understand and implement.

Limitations

- The program calculates the minimum multiplication cost but does not display the actual optimal parenthesization.
- Execution time can vary depending on the computer.
- Large numbers of matrices require more memory because of $O(n^2)$ space usage.

Algorithm

1. Start
2. Read the number of matrices.
3. Read the dimensions of all matrices.
4. Create a DP table.
5. Initialize the diagonal elements to 0.
6. Consider chains of length 2 to n.
7. Try every possible split position.
8. Calculate the multiplication cost.
9. Store the minimum cost in the DP table.
10. Display the minimum multiplication cost.
11. Display time complexity.
12. Display space complexity.
13. Display actual execution time.
14. Stop